Keyword Frames Are Cluster Samples: How a Third of an npm Ecosystem Stayed Invisible

Prachet Poddar

University of California, Los Angeles

prachetpoddar@gmail.com

Version 3.4, 8 September 2026

This work was conducted independently. It was not funded by, supervised by, or carried out under the auspices of any research program, and it should not be attributed to the University of California.

Summary

I set out to measure whether Model Context Protocol servers carry elevated supply-chain risk on npm. They do not. Getting to that answer took five wrong versions of this paper, and the errors are more useful than the result. They trace to two causes: how the sample was built, and a dependency resolver I never checked against the tool it was modelling.

A package registry's keyword search does not sample packages. It samples publishers. npm's search API orders by score and stops paginating near 5,000 results, so a frame built from it is a top-slice. Which packages fall in that slice is decided largely by who published them, because keyword tagging is a publisher-level habit rather than a per-package decision. Across twelve npm keyword ecosystems, publishers are entirely in or entirely out of a given co-occurring keyword far more often than independent tagging would produce: 73.2% against a 18.7% null for `mcp-server`, and above the null in all eleven that could be measured.

Three consequences follow, and each one invalidated a version of this paper.

The population has no single size. `keywords:mcp-server` returns 8,249 packages. `keywords:model-context-protocol` returns 35,607. `keywords:mcp` returns 70,922, and a verified sample confirms that filter is exact. Only 12% of the largest population carries the tag I built the frame on.

A third of the population is machine-generated and sits outside the frame. Four accounts hold 34.4% of the enumerated `mcp-server` population. One of them publishes packages with exactly six files and zero dependencies, standard deviation zero across every package sampled. Another has published 5,285 packages of which my frame contained 256, because it tags with `mcp` rather than `mcp-server`.

Enumerating the frame completely does not fix the frame. I took coverage from 63.8% to 95.1% by partitioning on maintainer, which changed two results and reversed one. It was still 95.1% of the wrong set.

The measuring instrument was wrong as well. Every dependency tree here comes from a reimplementation of npm's version selection, and I validated it against its own logic rather than against npm. Four defects survived that. Each was silent, each changed a published number, and every one of them fell out of the first comparison against a real `npm install`. The corrected resolver agrees with actual installs on

99.97% of nodes. Versions before 3.2 also gave a pre-correction agreement rate; no pre-correction validation run was saved, so that figure is not restated here. Two rows of Section 6 changed verdict as a result, and both had been reported on control arms whose dependency trees were an order of magnitude too small.

The findings that survive all of this are narrow and I state them as such. MCP servers ship license files more reliably than either control ecosystem, on both the corrected and the uncorrected rate. They publish exactly once more often than langchain servers do, on a coverage-matched comparison, though that result rests on the coverage correction and does not survive without it. Nothing I measured shows elevated supply-chain risk on any dimension a package registry exposes.

---

1. What was being measured

An MCP server is launched by a line in a client configuration file:

```
"filesystem": { "command": "npx", "args": ["-y", "@modelcontextprotocol/server-filesystem"] }
```

`npx -y` resolves from the public registry at launch, so the code that runs is whatever the registry serves at that moment. Nothing is pinned, no lockfile is produced, and the resulting tree appears in no manifest a scanner can read. Across 250 servers walked with the corrected resolver of Section 2.2, the median server installs 93 packages and the largest 618. A quarter install exactly one, and roughly two thirds install more than 50. 15.6% of trees contain a `preinstall`, `install` or `postinstall` hook, rising to 53.6% once a tree exceeds 120 packages.

Earlier versions of this section reported a median of 97 and a maximum of 589. Those came from the pre-correction walk and were left in place when Section 2.2 was rewritten; they are corrected here. Maximum depth was recorded only on the pre-correction walk, where it was 11, and is not restated.

These are properties of the launch mechanism rather than of any one package, but they are not independent of the frame: the 250 servers are drawn from stratum A, and the tail strata install exactly one package more often than stratum A does.

2. Method

2.1 Enumerating a keyword population

The npm search API returns at most 250 results per request and stops paginating near 5,000. For a population of 8,249 that gives 63.8% coverage, ordered by search score. Two things extend it and one does not.

Free-text term expansion. Adding a second term to the query (`keywords:mcp-server database`) reaches packages the unrestricted query cannot. A hundred terms lifted coverage to 79.5% and then saturated: the last ten terms added 26 packages.

Maintainer partitioning. `keywords:mcp-server maintainer:cyanheads` returns 121 against the unrestricted 8,249, so the qualifier is applied. Every package has a

maintainer and no maintainer's slice approaches the cap, so complete maintainer coverage implies complete package coverage. This took coverage to 95.1%.

`scope:` does not work. `keywords:mcp-server scope:pipeworx` returns 8,249, identical to no filter. npm accepts the qualifier and ignores it. A frame built on it would look partitioned and would not be. This is the third silently ignored query parameter I hit; Section 7 lists the others.

The maintainer crawl has a blind spot worth stating, because I fell into it. Maintainers are discovered from search results, and search is score-ordered, so a maintainer whose every package sits below the cutoff is never seen. Reading the full `maintainers` array from each packument, rather than the single publisher the search returns, surfaced 1,415 maintainers search never showed. That is what took coverage from 82% to 95.1%.

Final coverage is 7,838 of 8,238 in four strata:

Stratum	Reached by	Size	Share
A	score-ordered search	5,251	63.7%
B	free-text term expansion	1,289	15.6%
C	maintainer partitioning	1,298	15.8%
D	nothing	400	4.9%

Earlier versions of this table gave 5,250 / 1,290 / 1,293 / 405, which summed to 7,833 rather than the 7,838 stated above. The sizes here are the ones the released frame files reproduce. The correction moves no point estimate and moves two interval bounds by 0.1 point.

Strata B and C were sampled at n = 250 each and measured with the same code as stratum A. Population estimates are the size-weighted combination of the three observed strata, with bounds setting stratum D to all-clean and all-bad.

2.2 Dependency resolution

Trees are resolved by reimplementing npm's own selection rules, and the implementation is validated against real installs rather than against its own logic. For a range, npm-pick-manifest takes the version tagged `latest` when it satisfies the range, even where a higher version exists; failing that the highest non-deprecated satisfying version; failing that the highest satisfying version. Non-optional `peerDependencies` are installed, as npm 7 and later do, while peers marked optional in `peerDependenciesMeta` are not. A package already placed at a version satisfying a new range is reused rather than resolved again, so a second copy appears only on genuine conflict. Optional dependencies are included without evaluating `os` and `cpu`, making each tree an upper bound for any single machine. Non-registry specifiers are classified rather than dropped. Trees are keyed on (name, version), so a package legitimately present at two versions appears twice.

Each of these rules was established by comparing computed trees against the `package-lock.json` produced by an actual `npm install`, and each was absent from an earlier version of this study. Validation covers 75 distinct packages drawn after the corrections and spanning all three arms, on which agreement is 6,144 of 6,146 nodes. Thirty of those packages were involved in finding the defects above; on the remaining 45, which the resolver had never been debugged against, agreement is 3,790 of 3,790. The disagreement is confined to one package, `n8n-nodes-neo4j`, where npm installs `@types/node` and `dotenv` that this resolver does not place, and this resolver places an `openai` version npm does not. Two nodes missing and one invented, in a single tree.

Version 3.0 of this paper reported 9,351 of 9,355 nodes across 114 packages on two disjoint samples. That figure was wrong. The harness reads its random seed from the environment for the MCP arm but holds the two control arms' seeds fixed, so a second run redraws only one arm. I summed the two runs as though they were disjoint, which counted 39 packages twice. The figures above are the de-duplicated ones.

All resolution reads a single registry snapshot, re-fetched in one pass, whose timestamp is recorded in the released data. The package-metadata cache used previously never expired, so results drifted with the registry in a way nothing recorded.

2.3 Statistics

Proportions carry 95% normal-approximation intervals. Because publisher clustering is present, comparisons are made on a stratified cluster bootstrap that resamples publishers rather than packages, 4,000 resamples, symmetrically for both arms. Earlier versions used a design effect computed from realised cluster sizes; that estimator produced two wrong answers in this study and is retained only as the sensitivity check reported in Section 5.1. "GitHub Actions" is not treated as a publisher identity, since it is a CI label attached to packages from unrelated projects; those packages are treated as singleton clusters. Section 5 explains why that correction turns out to be small and why it would not be if the sampling had been done differently.

---

3. Result: the population has no single size

npm keyword	Packages
`mcp-server`	8,249
`model-context-protocol`	35,607
`mcp`	70,922

I verified the largest filter is exact before using it: 750 packages sampled from `keywords:mcp` at ranks 0, 2,000 and 4,750, and all 750 literally carry the keyword.

Anchored on the fully enumerated `mcp-server` frame, all 7,838 packages:

Also carries	Share
`mcp`	93.1%
`model-context-protocol`	64.6%
all three	64.2%

The reverse direction, sampled at six pagination depths in each larger population, is stable rather than depth-dependent:

Population	Carrying `mcp-server`, by depth
`model-context-protocol`	18.8, 23.6, 22.0, 20.8, 19.2, 19.6%
`mcp`	10.0, 12.4, 11.6, 11.2, 13.2, 15.2%

The two directions agree: 12% of 70,922 is about 8,500, and 93.1% of 8,249 is about 7,680. The stability across depth also means the top-slice bias is small for this particular ratio, so these estimates need no correction.

This study measured roughly one ninth of the packages that self-identify as MCP on npm. Every rate in Section 6 is conditional on a tagging convention.

4. Result: a third of the population is machine-generated

Four scopes hold 34.4% of the enumerated population, and the threshold does not matter: any cut between 25 and 250 packages per scope gives 33% to 37%.

Scope	Packages	Files per package	Direct deps	Unscoped name already taken
`@pipeworx`	1,257	**6, standard deviation 0.0**	**0 in every one**	3.3%
`@mseep`	1,065	3 to 274	0 to 16	**56.5%**
`@iflow-mcp`	255	5 in both sampled	0 to 2	4.6%
`@cyanheads`	115	47 to 154	3 to 7	23.5%

The last column asks, for each `@scope/name`, whether `name` without the scope already resolves to a package on npm that is not itself in this frame. It is measured on 120 packages per scope, released as `bulk_name_collisions.json`. Versions before 3.3 gave 1.1%, 10.1%, 14.9% and 0.9% in this column with no implementation behind them; the paper's own revision history records that the name-collision code was withdrawn in 1.0.2 and never released. Those four figures are withdrawn and replaced by the measured ones above.

The measure cannot distinguish republishing someone else's name from publishing your own package under both a scoped and an unscoped name, which is the likely reading of `@cyanheads`, whose examples are its own servers.

The correction changes the conclusion of this section, and in the direction that matters. These are three different things. `@pipeworx` is a template with zero variance whose names are almost all new, so it generates. `@mseep` is the opposite: more than half its unscoped names already exist, so it mirrors. The earlier claim that ninety percent of bulk output carries novel names holds for `@pipeworx` and `@iflow-mcp` and fails for the largest republisher. Generation and mirroring are both present, in different scopes, and the earlier text asserted only the first.

It arrives in bursts. Sampling 120 packages from each scope and reading first-publish dates from the packuments, `@mseep` published 70% of its sampled output on its three busiest days and `@pipeworx` 45% on its three. Versions before 3.3 gave 74% and 43% from a sample that was not preserved; the figures here are from a seeded redraw released as `bulk_burst_recovered.json`. A frame drawn before a burst and a frame drawn after describe different populations.

They sit outside the frame by construction. Bulk scopes are 16.7% of stratum A, 53.4% of stratum B and 78.3% of stratum C. The deeper the enumeration, the more of the population is machine-generated.

The clearest single case: `chatflowdev`, which publishes `@iflow-mcp`, has published 5,285 packages. My frame contained 256 of them. Its keyword distribution shows why: 3,303 tagged `mcp`, 1,546 `model-context-protocol`, 256 `mcp-server`.

5. Result: keyword tagging is a publisher-level property

For publishers with at least five packages, how often is a publisher entirely in or entirely out of the most common co-occurring keyword, against a Monte Carlo null that keeps publisher sizes and tags each package independently at the population rate:

Ecosystem	Publishers	Observed	Null	Ratio
embeddings	30	63.3%	4.7%	13.4
ai-tools	32	75.6%	11.8%	6.4
rag	46	66.5%	10.9%	6.1
langchain	22	73.6%	13.0%	5.6
mcp-server	74	**73.2%**	18.7%	3.9
eslint-plugin	38	34.7%	14.0%	2.5
babel-plugin	29	80.0%	43.6%	1.8
vite-plugin	32	76.9%	42.9%	1.8
gatsby-plugin	66	75.2%	51.5%	1.5
strapi-plugin	9	64.4%	43.3%	1.5
fastify-plugin	9	77.8%	51.0%	1.5

Every ecosystem clusters. The ratio is not comparable across ecosystems because the null depends on the base rate, so the intraclass correlation and the design effect it implies are the right statistics:

Ecosystem	ICC	Packages per publisher	Population design effect
mcp-server	0.280	**11.6**	**4.0**
ai-tools	0.619	5.2	3.6
babel-plugin	0.679	4.5	3.4
embeddings	0.432	5.1	2.8
rag	0.459	4.3	2.5
vite-plugin	0.431	4.2	2.4
strapi-plugin	0.557	3.4	2.3
langchain	0.456	3.6	2.2
gatsby-plugin	0.397	3.5	2.0
eslint-plugin	0.088	8.1	1.6
fastify, serverless	0.000	5.0, 2.9	1.0

MCP's publishers are not unusually consistent. They are unusually large. Its ICC of 0.280 is mid-range and lower than babel-plugin's 0.679. What makes its design effect the highest is 11.6 packages per publisher against 3.4 to 5.2 nearly everywhere else. Bulk publishing multiplies whatever clustering exists.

5.1 Where the clustering does and does not cost you

It is tempting to conclude that every interval in a registry-scale study should be widened fourfold. That is wrong, and I made the error before catching it.

A design effect is `1 + (m - 1) x ICC` where `m` is the mean number of sampled packages per cluster. The population has 11.6 packages per publisher, but a sample of 400 packages drawn from 5,250 has 1.7. Computing the design effect on the realised samples, for the actual outcomes rather than for keyword tagging:

Row	MCP	eslint-plugin	langchain
No license file	1.31	1.30	1.07
Declares, no text	1.32	1.30	1.07
Released last 30 days	1.37	1.02	1.00
Exactly one version	1.60	1.34	1.13

The two tree-shaped rows are absent because Section 6 reports them on the cluster bootstrap instead. Their design effects were computed on a resolver since found to be wrong, and recomputing a statistic this paper has stopped using would only invite a reader to use it.

Versions before 3.3 concluded here that intervals widen by 2% to 26% and that the design effect could not exceed 1.7 even at ICC = 1, because `m` was near 1.7. That

argument is wrong, and it is wrong for the same reason that retired this estimator from Section 6: `m` above is the arithmetic mean cluster size, and the correct quantity for unequal clusters is Kish's `m* = sum(n_i^2) / sum(n_i)`.

On the realised stratum A sample of 400, grouped by scope, there are 317 clusters with an arithmetic mean of 1.26 and a Kish `m*` of **8.26**, because one scope supplies 51 of the 400. The design effects that follow are 4.2 to 7.4, not 1.1 to 1.3. Scope is a proxy for publisher here, since the released sample records does not carry a maintainer field, so treat the magnitude as indicative and the direction as firm.

The table above is therefore retained only as a record of what the retired estimator produced. It is not evidence that clustering is cheap.

What the clustering costs is best read off the bootstrap, not off this table. The cluster bootstrap in Section 6 resamples publishers and so handles unequal cluster sizes correctly, and its z values are 25% to 45% smaller than the design-effect route on the same comparisons. That is the size of the cost, and it is consistent with a design effect well above 1.7 rather than with one bounded by it.

Publisher clustering damages frame construction as well as variance. It decides which packages you can see, and Section 3 is about that. It also widens the intervals more than an earlier version of this section allowed. The practical lesson is unchanged in its first half and withdrawn in its second: spend effort on enumerating the population, and do not compute a design effect from an arithmetic mean when a handful of publishers dominate the sample.

6. Result: the supply-chain measurements

Coverage-corrected over three observed strata for the MCP arm, with stratum D bounded. Every z below comes from the publisher cluster bootstrap of Section 2.3, which is the paper's primary estimator; earlier versions of this table printed design-effect values for rows 1, 2, 4 and 5, which was inconsistent with Section 2.3 and is corrected here.

The comparison each z makes needs stating plainly, because an earlier version of this preamble described it wrongly. The MCP column is the coverage-corrected population estimate. The control columns are uncorrected samples of their keyword frames. langchain is enumerated at 1,973 of 1,974, so a corrected MCP figure against langchain is coverage-matched. eslint-plugin is a 74% top-slice, so a corrected MCP figure against eslint-plugin is not, and comparing a corrected estimate against an uncorrected one is the error Section 7 identifies as this study's original fatal mistake. Where a row's separation against eslint-plugin depends on the correction rather than surviving it, that cell is not reported.

Rows 1, 2, 4 and 5 are package-level and were unaffected by the resolver correction. Rows 3 and 6 are tree-level and were re-measured:

#	Dimension	MCP	Bounds	eslint-plugin	langchain	z vs eslint	z vs langchain
1	Ships no license file	23.5%	[22.3, 27.2]	36.4%	50.0%	**−2.35**	**−5.37**

#	Dimension	MCP	Bounds	eslint-plugin	langchain	z vs eslint	z vs langchain
2	Declares a license, ships no text	22.0%	[21.0, 25.8]	36.0%	49.2%	**−2.58**	**−5.60**
3	Deprecated package in tree	18.0%	[17.1, 22.0]	27.5%	33.3%	−1.89	**−3.13**
4	Released in last 30 days	19.5%	[18.5, 23.4]	11.6%	16.4%	+1.07	+0.64
5	Published exactly one version	45.2%	[43.0, 47.9]	26.4%	21.2%	not reported	**+4.10**
6	Install hook in tree	15.7%	[15.0, 19.8]	3.3%	27.5%	**+3.62**	**−2.32**

Sensitivity to the coverage correction. The MCP column is corrected and the control columns are not, so it matters how much of each separation the correction is doing. Recomputing every comparison on MCP's uncorrected stratum A rate, which is a top-slice as both controls are:

#	Dimension	stratum A	z vs eslint	z vs langchain
1	Ships no license file	22.2%	−3.36	−6.86
2	Declares a license, ships no text	21.2%	−3.53	−6.94
4	Released in last 30 days	28.5%	+5.06	+3.43
5	Published exactly one version	25.2%	**−0.27**	**+1.04**

Rows 1 and 2 are unchanged in conclusion and are the results I would defend furthest. Row 4 separates on the uncorrected rate and not on the corrected one, which is the opposite of what an earlier version of this table reported, and I now treat it as unresolved. Row 5 is the row the correction carries entirely: at 45.2% it separates from both controls, and at stratum A's 25.2% it separates from neither. `mcp-confound-`

`tests.json` in the released data records that comparison directly, with the note that 21% to 26% single-version publication is the npm baseline once frames are coverage-matched.

I therefore report row 5 against langchain only, where both arms are enumerated and the comparison is coverage-matched, and not against eslint-plugin, where it would be a corrected estimate against a 74% top-slice.

Two results are firm: MCP servers ship license files more reliably than either control, on both the corrected and the uncorrected rate. Single-version publication at 45.2% is higher than langchain's 21.2% on a coverage-matched comparison, and that finding depends on the coverage correction rather than surviving without it. Rows 3 and 4 separate against one control and not the other, and I report them as unresolved, though row 3's point estimate and its size-adjusted odds ratios, 0.27 against eslint-plugin and 0.32 against langchain, both favour MCP.

Row 6 is no longer a null. Earlier versions attributed install-hook exposure to tree size rather than to MCP, on control arms whose median trees were 8 and 6 packages against MCP's 96. That gap was an artifact of excluding peer dependencies, which eslint plugins and n8n community nodes rely on heavily and MCP servers barely use. With peers resolved, mean tree size is 91.8, 86.3 and 79.7 across the three arms and the rates still differ by a factor of eight, so size does not explain them. MCP sits above eslint-plugin and below langchain, and both separations survive size stratification, with Mantel-Haenszel odds ratios of 5.05 and 0.31.

Nothing here shows elevated supply-chain risk. The one dimension where MCP is clearly worse than a control is install hooks against eslint-plugin, and eslint plugins are pure-JavaScript packages that would not be expected to compile anything. Against langchain, the closer comparison in both purpose and dependency shape, MCP is significantly lower on install hooks and on deprecated packages in tree. The direction of every correction made during this study favoured MCP, which is the most comfortable possible outcome and therefore the one deserving the least trust; it is the reason the resolver was validated against real installs rather than against itself.

6.1 The composition caveat

Every rate above is a property of a population that is one third machine generated. Excluding the bulk scopes, single-version publication falls to 21.3%, level with langchain's 21.2%, and the tail's missing-license rate rises to 41.8% and 44.4%, above eslint-plugin's 36.4%. Both headline findings are composition effects as much as they are properties of MCP servers.

I report the pooled figures as the population values, because the generated packages are genuinely in the population and a reader deploying an MCP server picks from that population. But a reader asking what a hand-written MCP server looks like should use the non-bulk figures, and the two questions have different answers.

7. Sampling errors, mine and available to anyone

Six are documented here because each one produced a result I published and withdrew.

The search frame is a top-slice. Comparing a 2.4% top-slice of `keywords:cli` against 64% of MCP produced z = +7.32 for an effect that does not exist. Match coverage fractions across arms, or enumerate.

Fixing it in one place is not fixing it. Version 1.0.0 rebuilt three rows of its table on coverage-matched frames, left four rows on the invalid one, and described the whole table as matched. Re-running those four changed two and reversed one.

Silently ignored parameters. Docker Hub accepts `ordering=-pull_count` and ignores it, returning an arbitrary 100 of 245 repositories with a Gini of 0.16 where a power law was expected. npm accepts `scope:` and ignores it. Verify that a filter took effect before building on it.

Skipping on rate limits. A run that skipped HTTP 429 lost 55 of 328 Docker repositories, running alphabetically from `scorecard` to `zscaler`, which contained six of the ten largest. A later maintainer crawl terminated with 155 maintainers unprocessed, all beginning x, y or z. Rate limiting is correlated with alphabetical position, and alphabetical position is correlated with nothing until it is.

Symmetry is not sufficient. Peer dependencies were excluded from every arm, which looked safe because the exclusion was identical. It was not, because the arms differ in the thing excluded: eslint plugins declare `eslint` as a peer, MCP servers put the SDK in `dependencies`. An identical procedure produced control trees an order of magnitude smaller than the treatment trees, and a row was published as a null on that basis. Matching a procedure across arms is not the same as matching its effect.

I never checked the tool I was modelling. The resolver was validated against its own logic for the entire study and never once against `npm install` itself. Four defects survived: taking the highest satisfying version where npm prefers the one tagged `latest`, taking deprecated versions where npm avoids them, omitting auto-installed peers, and resolving every edge independently where npm reuses an already-placed version. Each was silent, each changed published numbers, and every one fell out of the first comparison against a real install. A fifth issue was not a defect but had the same effect: a package-metadata cache that never expired, so results drifted with the registry in a way nothing recorded. Reimplementing a package manager's resolution is a reasonable thing to do and an unreasonable thing to trust.

8. Two results that never depended on the frame

PyPI usage concentration. The PyPI arm was drawn from the simple index, which enumerates completely, so it never had a frame problem. Across 221 PyPI-distributed servers with 203,446 combined monthly downloads, the Gini coefficient is 0.9611 and one package takes 63.6% of installs. A finding that survived every control I had, PyPI servers missing license metadata at 32.8% against an 18.0% baseline, z = +3.78, falls to **1.9%** when weighted by downloads. Maintenance activity is not a proxy for adoption: packages with four or more releases and activity in the past year showed 31.1% missing, and I predicted the weighted figure would land between 20% and 35%.

Docker's curated catalog. All 328 catalogued entries, fetched one repository at a time. 182 have a published image and 146 do not. Gini 0.7094, top item 11.4%, top five 38.6%, median 33,892 pulls, no image with zero pulls. A curated channel is skewed but not degenerate, which suggests the extreme tail is a property of open publication rather than of MCP. Recomputing from the released per-repository records gives the

same figures, with a median of 34,009 rather than 33,892 because the stored summary takes the lower of the two middle values on an even count.

9. Limitations

The population is defined by a keyword and Section 3 shows that choice moves the denominator by a factor of 8.6. Everything in Section 6 is conditional on `mcp-server`.

405 packages were reached by no method. Bounds set them to both extremes.

Control arms are not enumerated to 95%. langchain is complete at 1,973; eslint-plugin is a 74% frame carrying the same bias MCP's frame had, so eslint-plugin comparisons are provisional and only langchain comparisons are clean.

The ICC estimates in Section 5.1 are poorly identified, since most sampled publishers contribute one package. The design effects are bounded above by the mean cluster size regardless, which is why the conclusion survives.

Trees are keyed on (name, version) and reuse an already-placed version when it satisfies a new range, matching npm on 6,144 of 6,146 nodes over 75 packages, and on 3,790 of 3,790 over the 45 of those with no part in finding the resolver defects. Agreement here is the share of npm's nodes this resolver also produces, so it does not penalise nodes produced that npm does not install; over the same 75 packages there is one such node. The residual is npm nesting a package at two versions where this resolver keeps one, so tree contents undercount in that specific respect. Optional dependencies are included without evaluating `os` and `cpu`, so trees are an upper bound for any single machine in the other direction.

The MCP arm is coverage-corrected to 95.1% enumeration and the control arms are not, so the corrected MCP figures and the control figures are not directly commensurable. Section 6's table reports the corrected MCP figure against uncorrected controls, and gives the stratum A recomputation alongside it so a reader can see which separations depend on the correction. Versions before 3.2 claimed the z values were drawn from stratum A. They were not, and the claim is withdrawn.

All resolution reads one registry snapshot. Re-running against a later snapshot will move the tree-level rows, since deprecations and publications accumulate.

I measured license and maintenance metadata. I did not measure malware, typosquatting outcomes, prompt injection, tool poisoning, or runtime capability scope, which is where most published MCP security work sits. None of this says anything about those dimensions.

10. What this means

For anyone assessing MCP supply-chain risk on the dimensions a package registry exposes: this ecosystem is not worse than its neighbours.

For anyone measuring a package ecosystem: a keyword frame is a cluster sample of publishers. Enumerate the population before computing anything over it, verify that every filter you use took effect, and report which tag you chose, because that choice is a larger source of variation than anything you are likely to measure.

The practical consequence for MCP specifically is that the published population and the installed population are almost disjoint, that a third of the published population is

machine-generated, and that no static frame describes it for long. Whether the absence of visibility into what an agent installs is a problem anyone needs solved is not a question measurement can answer.

11. Data and code

Everything is released, including the superseded versions of this paper that still exist as files, both invalidated datasets, and the raw per-package records behind every corrected figure:

> https://github.com/prachetpoddar/mcp-supply-chain-study
>
> Archived at DOI 10.5281/zenodo.22641812.

Cite that DOI. It is the concept DOI, which resolves to the most recent archived version, so it does not go stale as this paper is revised.

The version DOIs are more specific and, in one case, misleading. 10.5281/zenodo.22641813 archives version 1.0.0 under its original title, "Measuring the MCP Supply Chain: Eight Null Results and a Population That Is Not the Population", whose findings are superseded here. 10.5281/zenodo.22664719 is labelled version 3.3 but archives the version 3.1 tree, because the `v3.3` git tag was placed on the version 3.1 commit and Zenodo archives whatever the tag points at. It contains neither the Section 6 correction nor the lock format proposal. Versions 1.0.1, 1.0.2 and 2.0 were never archived, and versions of this paper before 3.3 stated that 2.0 had its own DOI, which was not true.

Code is Apache-2.0, data CC-BY-4.0. The resolver uses only the public npm, PyPI and Docker Hub APIs. No third-party package source is redistributed.

12. Revision history

1.0.0 Seven null results against a control frame that Section 7 shows to be invalid for four of them.

1.0.1 Re-measured rows 1 to 4 against coverage-matched frames. Two changed, one reversed sign. Withdrew the claim that MCP servers publish less often than comparable packages.

1.0.2 Enumerated the population to 95.1%. Row 5, a null in both previous versions, became a 45.2% versus 21.2% difference. Withdrew the name-collision figures, whose implementation had never been released.

2.0 Found that a third of the enumerated population is machine-generated by four accounts, that keyword tagging is a publisher-level property across eleven ecosystems, and that the keyword chosen moves the population size by a factor of 8.6. Corrected the design-effect error introduced in draft: the realised effect is 1.0 to 1.6, not the population value of 4.0. Restructured around the frame problem rather than around the risk question.

3.0 Validated dependency resolution against real `npm install` output for the first time. Four defects surfaced, none visible from inside the implementation: npm prefers the version tagged `latest` when it satisfies a range, avoids deprecated versions when a non-deprecated one satisfies, auto-installs non-optional peer dependencies from npm 7

onward, and reuses an already-placed satisfying version rather than resolving each edge independently. Agreement with real installs after the fixes is 99.97% of nodes. Two rows of Section 6 changed verdict. Re-resolved the corpus against a single dated registry snapshot, the previous package-metadata cache having never expired.

3.1 Corrected the validation figure. Version 3.0 reported 9,351 of 9,355 nodes across 114 packages on two disjoint samples. The harness reads its seed from the environment for the MCP arm but holds the control arms' seeds fixed, so the second run redrew only one arm; I added the runs together as though they were disjoint and counted 39 packages twice. De-duplicated, it is 6,144 of 6,146 nodes across 75 packages, and 3,790 of 3,790 across the 45 with no part in finding the defects. The strict figure is stronger than the one it replaces. The error was in how I combined the runs, not in the resolver.

3.2 Three corrections found by adversarial review before publication. The Section 6 z values for rows 1, 2, 4 and 5 were design-effect figures, while Section 2.3 states the cluster bootstrap is the primary estimator and the design effect is retained only as a sensitivity check; all six rows now use the bootstrap. Under it, row 4 no longer separates from either control and is moved to unresolved. The Section 6 preamble and Section 9 both claimed the z values were computed on MCP's stratum A rate; they were computed on the coverage corrected rate, and the claim is withdrawn rather than the numbers changed. The stratum A recomputation is now reported alongside, which shows that row 5 depends entirely on the coverage correction, so row 5 is reported against langchain only, where both arms are enumerated, and no longer against the eslint-plugin top-slice. Section 1's tree statistics were pre-correction figures presented as post-correction, and are corrected from a median of 97 and a maximum of 589 to 93 and 618. The stratum table summed to 7,833 against its own stated 7,838 and now carries the sizes the released frame files reproduce. The overlap between the two validation runs is 39 packages rather than 38. The residual disagreement was described as one package nested at two versions; it is two nodes npm installs and this resolver does not, plus one node this resolver produces and npm does not, in a single tree.

3.3 A pass over every quantitative claim against the released data files, prompted by two independent adversarial reviews. Six changes.

Withdrew the pre-correction agreement rate of 64.6% quoted in versions 3.0 and 3.1 and, before them, in version 2.0. The four resolver defects and the 99.97% post-correction figure are unaffected and are reproducible from the released per-package validation records. The pre-correction figure is not: no validation run against real installs was preserved from before the fixes, so there is no artifact behind it and it should not have been reported as though there were. A reader comparing versions will find the number gone rather than changed, which is the intended reading.

Section 4's name-duplication column gave 1.1%, 10.1%, 14.9% and 0.9% with no implementation behind them; the revision history above records that the name-collision code was withdrawn in 1.0.2 and never released, so reinstating those figures in 3.0 was an error. They are replaced by a fresh measurement, released as `bulk_name_collisions.json`, which gives 3.3%, 56.5%, 4.6% and 23.5% and reverses part of the section's conclusion: `@mseep` mirrors rather than generates, and the claim that ninety percent of bulk output carries novel names holds for two scopes and fails for the largest republisher.

Section 4's publication-burst percentages came from a sample that was not preserved. Redrawn from the packuments under a fixed seed and released as `bulk_burst_recovered.json`: 70% and 45% on the three busiest days, against the 74% and 43% previously reported.

Section 8's Docker statistics were withdrawn during this revision and are restored. The withdrawal was my error: I searched one working copy, did not find `mcp_docker.json` or `conc_docker.json`, and concluded the run had not been kept. Both files exist in the study folder, and the released per-repository records reproduce every figure: 182 published images of 328 catalogued, Gini 0.7094, top item 11.4%, no image with zero pulls. Reporting data as missing on the strength of one location is the same class of error as reporting a number without an artifact, and it is worth recording that it happened while correcting the opposite mistake.

Section 5.1 computed its design effects from an arithmetic mean cluster size where Kish's formula is required for unequal clusters, which is one of the two errors that retired this estimator from Section 6. On the realised stratum A sample the Kish `m*` is 8.26 against an arithmetic mean of 1.26, so the design effects are 4.2 to 7.4 rather than 1.1 to 1.3, and the section's conclusion that clustering costs little variance is withdrawn. The cluster bootstrap, whose z values are 25% to 45% smaller than the design-effect route, is the evidence for what the clustering actually costs.

The abstract said publishers exceed the tagging null in "all twelve" ecosystems. Eleven were measurable; the twelfth had too few multi-package publishers to measure and was dropped by the analysis code without being dropped from the prose.

The release assembler did not run. It referenced `paper/`, `data/` and `code/` subdirectories that do not exist in a flat working folder, under `set -euo pipefail`, so it aborted on its first copy and produced no release at all, and it named the version 1 paper. It now runs, ships the current paper and both superseded ones, and copies the per-package validation records that make the validation figure reproducible. The README, `CITATION.cff` and `.zenodo.json` described version 1.0.2 under the old title and are rewritten.

3.4 Corrected the archive references. Version 3.3 cited 10.5281/zenodo.22664719 as the archive of itself. That record is labelled version 3.3 but contains the version 3.1 tree, because the `v3.3` tag was placed on the version 3.1 commit, and a Zenodo record's files cannot be replaced after minting. The paper now cites the concept DOI, 10.5281/zenodo.22641812, which resolves to the most recent version and does not need changing at each revision. Also released the result files Sections 5 and 6 depend on, which had never been published: `cluster_bootstrap.json` and `design_effect_corrected.json` carry the z values Section 6 reports, `publisher_clustering.json` and `publisher_icc.json` are the whole of Section 5, and the per-package validation records make the 6,144 of 6,146 figure reproducible rather than requiring a reader to sum two arm summaries, which reproduces the figure revision 3.1 retracts. The paper claimed its data was released; for those sections it was not.